Design Spotify Trending Songs

功能性需求（Functional Requirements）

1. 服務應該能夠從使用者端收集收聽指標（listening metrics）。
2. 服務應該能夠依不同維度（dimensions，例如 country、genre）提供前 K 名（top K，例如 100）歌曲。

非功能性需求（Non-Functional Requirements）

3. 在每小時 / 每日結束後，系統應該能夠盡可能快速地（ASAP）產生並提供 top K songs。
4. Spotify 擁有約 7 億 MAUs（Monthly Active Users，月活躍使用者）以及 1 億 tracks。
5. 資料蒐集需具備高準確性（accurate data collection），避免 over-counting 與 under-counting。

API 設計（API design）

1. 傳送播放事件

```
POST/tracks/events
{
  event_id,
  session_id,
  track_id,
  position_ms,
  state,
  ...
}
```

- `event_id`（事件 ID）：每一筆播放事件的唯一識別碼，用於去重（deduplication）。

buildmoat.org

- `session_id`（播放工作階段 ID）：代表使用者一次連續的收聽行為，用來輔助判斷有效播放。
- `track_id`（歌曲 ID）：被播放的歌曲識別碼。
- `position_ms`（播放位置，毫秒）：目前播放到的時間點。
- `state`（播放狀態）：例如 play、pause、stop 等。
- 其餘欄位（...）：可包含裝置、網路、地區等輔助資訊。

2. 依指定維度取得過去一小時的 Top K 歌曲

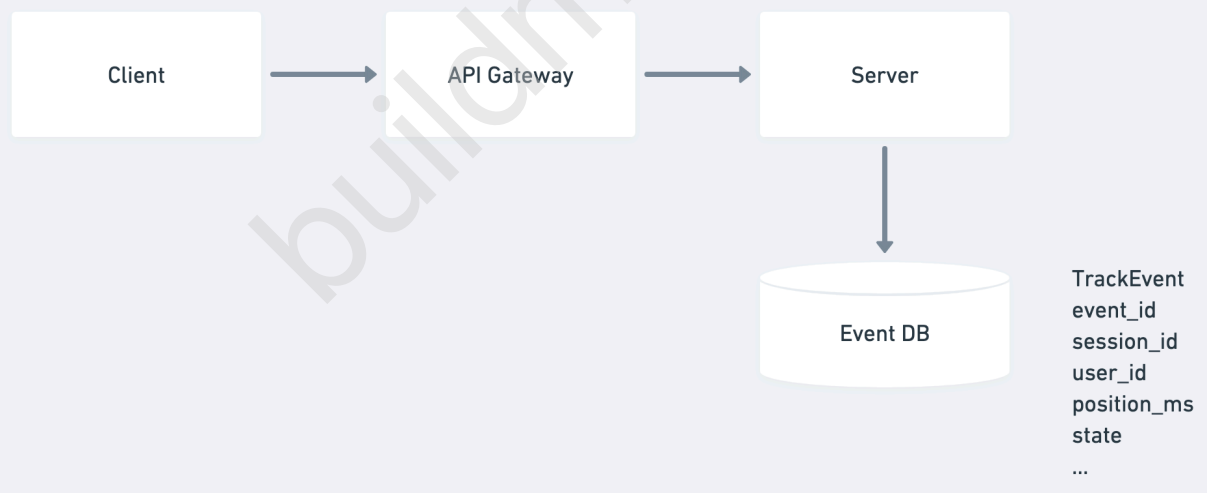
```
GET/tracks/top_tracks?dim={dim}&num_tracks={num_tracks}
```

- `dim`（dimension，維度）：例如 country、genre 等，用於分群統計。
- `num_tracks`（歌曲數量）：回傳的 top K 數量，例如 100。

此 API 會回傳指定維度下，於過去一小時內收聽次數最高的歌曲清單。

High-Level Design

1. 事件回報流程（Event reporting flow）



- 使用者裝置（User device）會以固定頻率（例如每 5 秒）將收聽事件（listening events）送至後端伺服器（backend server）。
- Server 會將每一筆事件寫入事件資料庫（Event DB）。由於此模式下的寫入吞吐量非常高，資料庫可選用 Dynamo 或 Cassandra。
- 為了節省網路資源（network resource），可以考慮將事件進行批次送出（batching），例如每累積 5 筆事件才送出 1 個 request。

buildmoat.org

- 若使用者離線（offline），可先將事件暫存在本地資料庫（local DB，例如 SQLite），待重新連線後再一次 flush 至 server。

2. 服務需能依不同維度提供 Top K 歌曲

下一個需求是：我們需要能提供過去一小時 / 一天的 top K songs。查詢邏輯可拆成兩個階段：

1. 查詢過去一小時內所有歌曲的 aggregate counts。
2. 對第 1 步的結果進行排序，取出前 K 筆。

這類查詢本質上都是 **超大型 table scan**。

以 7 億 MAUs 為例，假設其中 20% 的使用者每天會聽音樂 1 小時，則每小時需處理的事件數量為：

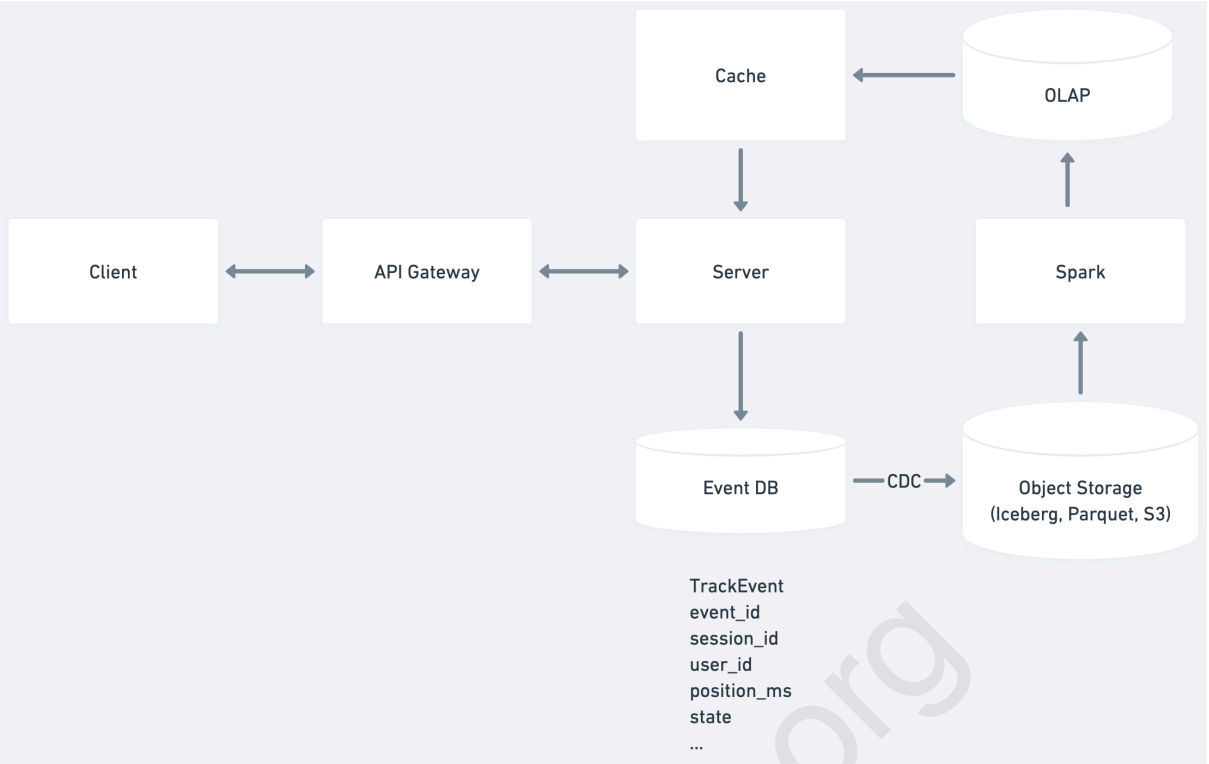
```
700M * 20% * 3600（每小時秒數） / 5（每 5 秒 1 筆事件） / 24（每日小
= 約 4.2B events / hour
```

也就是說，每小時需要掃描約 42 億筆事件。

如果只是單純讓一組 workers 每小時直接掃 DB，這在實務上是不可行的。

- 傳統 OLTP DB（online transactional processing）是 row-based，並針對 point lookup（依 key 取單列資料）做最佳化，無法在時限內完成這類掃描，更不用說 query timeout 本身就會阻止這種不合理的昂貴查詢。
- 即使技術上能跑完 table scan，讓如此昂貴的分析流程與線上工作負載（此處為 event writes）競爭同一組資源，也極容易拖垮整體系統，通常被視為 anti-pattern。

buildmoat.org



因此，我們需要引入一個**專門用於分析查詢的資料庫**：OLAP DB（online analytical processing）。

OLAP DB 的一個核心特性是採用 column-oriented storage（相較於 OLTP 的 row-oriented）。

這代表：

- 查詢時只需讀取相關欄位
- 欄位層級可進行高效壓縮

這些特性讓 analytical queries 具備極佳效能。

將 raw data 轉換成 OLAP 的其中一種常見方式，是透過 Spark batch jobs。

資料處理流程

1. Event DB 與 CDC connector（change data capture）整合，將每一筆 change log 發送到 message queue（例如 Kafka），並寫入 data lakehouse 的 sink。
    - 通常會有額外的處理程序（Flink / Spark）將 raw data 整理後寫入 object storage（例如 S3），並採用 Apache Iceberg 這類 table format。
        - 由於 raw data 是 append-only events，Iceberg 非常適合處理 deduplication、snapshot versioning、schema evolution 等需求。
2. Spark batch jobs 會定期（通常最快每 10 分鐘）從 data lakehouse 讀取資料，將 event counts 聚合到每分鐘粒度，並將結果寫入 OLAP DB。

buildmoat.org

- 在底層，Spark 採用 map-reduce 模式，將檔案切分為多個 chunk 並行處理，最後再將多台機器的結果聚合成一個整體輸出。
- 這裡需要注意的是：data lakehouse 中的資料雖然以 Parquet（column-oriented format）儲存，但 Parquet 本身只是一種檔案格式，存放在 object storage（如 S3）上，並不具備 index、低延遲查詢、或 SQL 介面等能力。Spark 在這一步的主要工作是**大規模聚合運算**（將數十億筆 raw events 彙總為每分鐘的 play counts），而非單純的格式轉換。聚合後的結果寫入 OLAP DB，才能支援後續 worker 的低延遲分析查詢。

3. 當資料進入 OLAP 後，即可高效地支援 analytics queries。 由於查詢模式相對固定（例如「過去一小時 / 一天的 Top K songs」），我們也可以在回傳給使用者前，先對查詢結果進行快取（cache），進一步降低 DB 負載。

深入探討（Deep-Dives）

1. 我們要如何將資料新鮮度（data freshness）進一步降低到 1 分鐘以內？

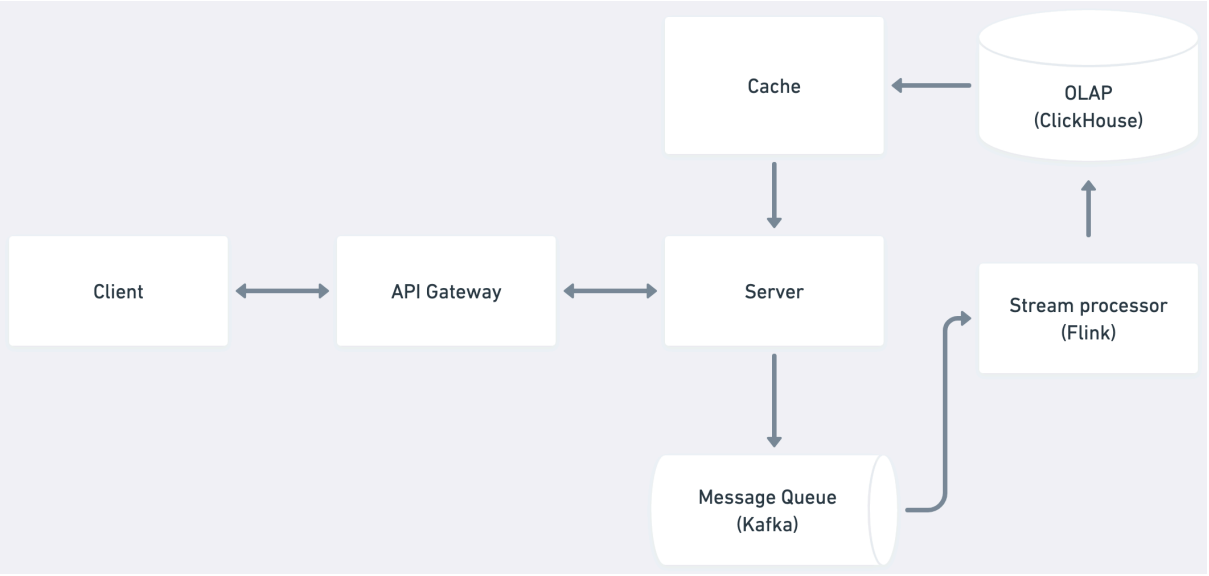
Batch solution 在「數分鐘內提供聚合資料」這件事上表現良好，但由於 batch job 的本質限制，其執行頻率存在下限：

- 啟動成本（startup overhead）
- 查詢規劃（query planning）
- 檔案 I/O
- JVM / GC 與 cache warmup

多數 scheduler 對單一 job 的最小排程間隔為 1 分鐘。實務上，考量 batch job 的整體 overhead，較合理的做法是每 10 分鐘跑一次。再加上 CDC 流程將資料從 OLTP 匯入，以及 batch job 後續將資料寫入 OLAP，代表我們通常需要在整點結束後等待 10–20 分鐘，才能取得最新 partition 的資料。

這對 top songs 這個 use case 來說通常是可以接受的。但如果我們希望進一步優化，讓系統能在 1 分鐘內提供最新資料，就必須引入 stream processing。

buildmoat.org



為了即時處理 raw data 並近即時更新聚合結果，我們可以採用以下架構：

- 我們移除了 OLTP，讓 server 直接將事件發佈到 message queue（例如 Kafka）。
- 使用 stream processor（例如 Flink）從 queue 中消費事件，逐筆處理，例如 deduping、sessionizing，並將有效的播放事件發佈為 PlayFact events 寫入 OLAP。
- 透過 Flink 進行**秒級（或數秒）微批聚合**後再寫入，延遲可控制在數百毫秒至數秒之間，大幅降低 OLAP 壓力，同時保留即時性。
- OLAP 可針對不同時間粒度（例如 1m、1h、1d）預先計算 materialized view。
- 另有一個獨立 worker 以秒級頻率查詢 OLAP，取得最新的 top K songs 並刷新 cache。
    - 例如：過去 1 小時的 top K 使用 1m 粒度，過去 1 天使用 1h 粒度。
- 最終的 top K 結果直接從 cache 回傳，以確保低延遲並避免大量重複查詢打到 OLAP。

整體 end-to-end 延遲應該能夠控制在 1 分鐘內（多數情況甚至可低於 10 秒）。

Stream processing 之所以比 batch 快，原因在於：

Batch（即使是很小的 batch）每次執行都必須付出以下成本：

- Cluster / job 啟動（executor allocation、JIT warmup）
- Planning 與檔案列舉（object store LIST、small-file overhead）
- 讀取資料、shuffle、寫入結果
- Table commit / manifest 更新（Delta / Iceberg / Hudi）或 warehouse load 步驟
- Scheduler 的保護間隔（cron / Airflow / Jobs）以避免 job 重疊

Streaming（Flink / Spark Structured Streaming）則只需付出一次大多數成本：

buildmoai.org

- 長時間存活的 operators 將 state 保存在 memory 或 RocksDB
- Kafka 提供連續的 incremental input（無需檔案列舉）
- **Checkpoints** 與 **exactly-once / upsert** commit 會在背景以短間隔執行
- 可在計算完成後立即推送部分聚合結果（例如 PlayFacts 與 top-K deltas）

2. 我們要如何計算「有效播放次數」（effective play counts）？

接下來深入探討如何定義每首歌的 play counts。若只是單純對每首歌計算 unique events，勢必會 over-count。

例如：使用者播放一首歌後，立刻 seek 到最後幾秒，接著按下一首。這種情況顯然不應算作一次有效播放。

Spotify 對 stream 的定義如下：

```
Counted when a listener plays your song for at least 30 secon
```

其中一種實作方式是：在 stream processor 中，為每個使用者的每個 track 維護一個 per-session state。在每個 session 中，維護一個 `accumulated_ms` 欄位。

當使用者播放某首歌時，裝置會持續送出具有相同 `session_id` 的事件；當 `accumulated_ms` 超過門檻（30 秒）後，stream processor 便會產生一個代表有效播放的事件（稱為 PlayFact event）。

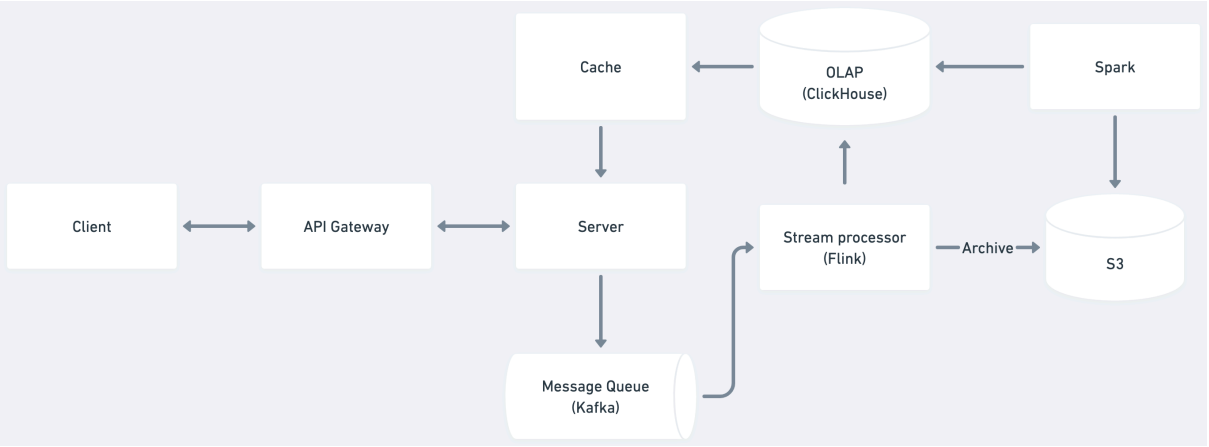
範例時間線如下：

- t=0：點擊 Play → `event_id=A`、`session=S1`
- t=10 / 20 / 30：heartbeat events → `event_id=B/C/D`、`session=S1`
- t=30：達標 → 為 S1 計算 1 次 play
- t=75：使用者點擊 Replay → `event_id=E`、`session=S2` → 可再次達標並計數

這只是避免 over-counting 的其中一個簡化示例。實務上，真正的實作會更複雜，並需要額外的 post-processing 來偵測人為刷流（artificial streaming）。

事實上，有許多 abuse detection analytics 無法即時完成，因此通常仍需保留 batch layer，在離線情境下重新處理資料，並與 serving data 進行 reconcile。

buildmost.org



3. 我們要如何將系統擴展到可承受事件吞吐量？

在前述分析中，我們得出每小時約 4.2B events，換算下來約為 **1.2M events/sec**。即使透過 batching 送出事件，系統仍需每秒處理數十萬筆事件。

- **Server**：Server 為 stateless，因此可輕易進行 horizontal scaling。請求可被隨機路由至任一 instance，多數雲端平台也提供 auto-scaling，可依 CPU 或 memory 使用率自動擴縮。
- **Streaming**：如前所述，每個 Flink worker 需要維護 session state。為了水平擴展，可使用 `session_id` 作為 partition key。由於每個 session 只包含有限事件數，這樣的 partitioning 能將負載相對平均地分散到各個 partition。
- **OLAP**：OLAP 也需要透過增加節點來進行水平擴展。一種可行的 sharding key 是 `dim`，因為查詢通常會依該欄位進行 filter 與 group by。 將相同 `dim` 的資料集中於同一個 node，有助於提升查詢效率。 需要注意的是，僅以 `dim` 分片容易產生 hot shard。例如在 country 這個 dim 下，US 的事件量可能遠高於 Taiwan。 一個解法是為預期的 hot shard 加上 sub-sharding，例如將 US 拆分為 `US-1`、`US-2` … `US-N`，讓美國的事件分散在多個 shard 上，而非集中於單一 shard。

pulumpat.org